



NAVAL
POSTGRADUATE
SCHOOL

AE2440: Introduction to Scientific Programming

Why Differential Equations Invented the Computer



BLUF

- **Modeling**, building useful abstractions of real systems, is the core of what engineers do to understand, assess, and design solutions.
 - **Differential Equations** are the means for expressing many useful engineering models
- **Computing** is built on a 300-year-old idea: approximate the continuous world with discrete, incremental steps.
- The history is defined along two fronts:
 - **Techniques**: faster tools that allow bigger models:
 - Minutes to nanoseconds. Problems once impossible become routine.
 - **Interfaces**: more abstract tools that widen who can build them
 - Pencils → punch cards → assembly → FORTRAN → MATLAB → AI
- The need to integrate DEs faster has driven the last 300 years of computing innovation
- Solving DEs is the killer app for computing — not by coincidence, but by design.
- The historical trend: closing the gap between continuous reality and discrete models



The Calculus Wars (1660s–1700s)

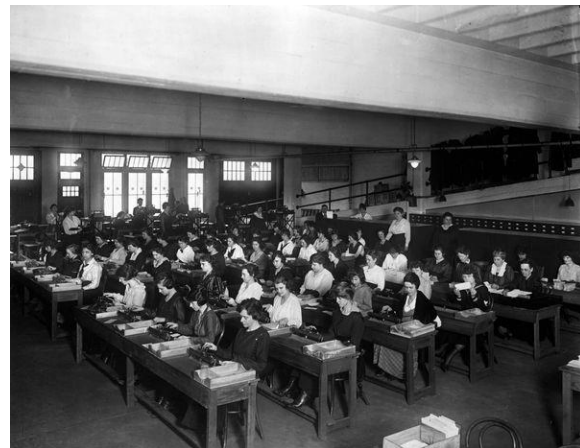
- Newton & Leibniz — Why Calculus?
 - Newton needed to explain why planets move in ellipses — Kepler had described the orbits but not the cause. Required a way to reason about instantaneous velocity and acceleration.
 - Leibniz approached the same problem from geometry
 - Both arrived at the same insight: physical laws are naturally statements about rates of change. Writing $F = ma$ as a differential equation and solving it is the direct line from their work to every numerical solver in use today.





The 18th-Century Analysts

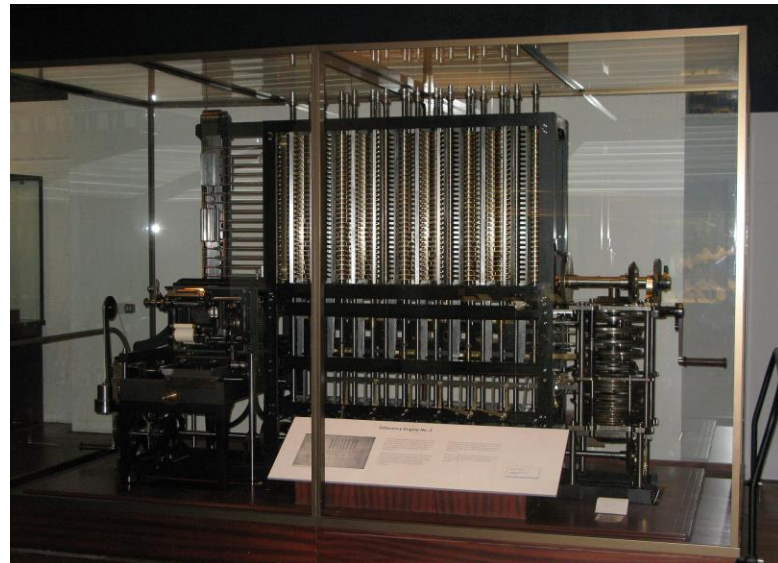
- Euler (1707–1783)
 - Problem: pendulums, fluid flow, celestial perturbations — equations with no closed-form solution.
 - Answer: if you can't solve it exactly, step forward numerically.
 - Euler's method (1768) — written blind, dictated to a secretary.
- Laplace (1749–1827)
 - Problem: Newton solved two bodies cleanly. The real solar system has eight planets all pulling on each other. Is it stable?
 - Answer: convert the differential equation into an algebraic one.
 - The Laplace transform makes whole classes of ODEs suddenly tractable.
- Fourier (1768–1830)
 - Problem: Napoleon wants to know how heat moves through cannon barrels and fortification walls.
 - Answer: represent any function as an infinite sum of sines and cosines.
 - Paper rejected by Lagrange and Laplace as insufficiently rigorous. Published anyway, 1822.





Babbage, Lovelace, and the Mechanical Dream (1830s–1840s)

- The problem: Mathematical tables were computed by hand, full of errors, and ships navigated by them. Wrong tables meant shipwrecks.
Babbage: *I wish to God these calculations had been executed by steam.*
- Difference Engine (1822–1842)
 - Gears and levers grind out table values mechanically. No human arithmetic, no human error. Funded. Redesigned repeatedly. Never finished.
 - His original drawings were followed exactly in 1991. It works perfectly.
- Analytical Engine (1837 onward)
 - Memory, processor, conditional branching.
 - A complete general-purpose computer.
 - Also never built. An [active project](#) exists today to finally do it.
- Ada Lovelace
 - Daughter of Lord Byron, raised to be mathematical as a deliberate antidote to her father.
 - Writes the first machine algorithm at 27. Then states the limit no one else had seen:
 - *The Analytical Engine has no power of originating anything. It can only do what we know how to order it to perform.*
 - First statement of computability. 1843.





Human Computers – Assembly Line of Arithmetic (1880s–1940s)

- Break the calculation into steps. Hire a room full of people. Run them in parallel. Pass results down the line.
- WWII industrializes it.
 - Every artillery piece needs firing tables — trajectory at every angle, every wind condition, every atmospheric variation. Each trajectory is an ODE. Each table takes months.
 - The Army recruits math majors — predominantly women — from universities across the country. Over 100 working simultaneously at the University of Pennsylvania.
 - Still not fast enough.
 - This is the problem that builds ENIAC.
 - Every trajectory they computed by hand is exactly Euler's method — evaluate the slope, step forward, repeat. With pencils. For months.



"Computers" doing ballistic calculations at UPENN, which would build ENIAC

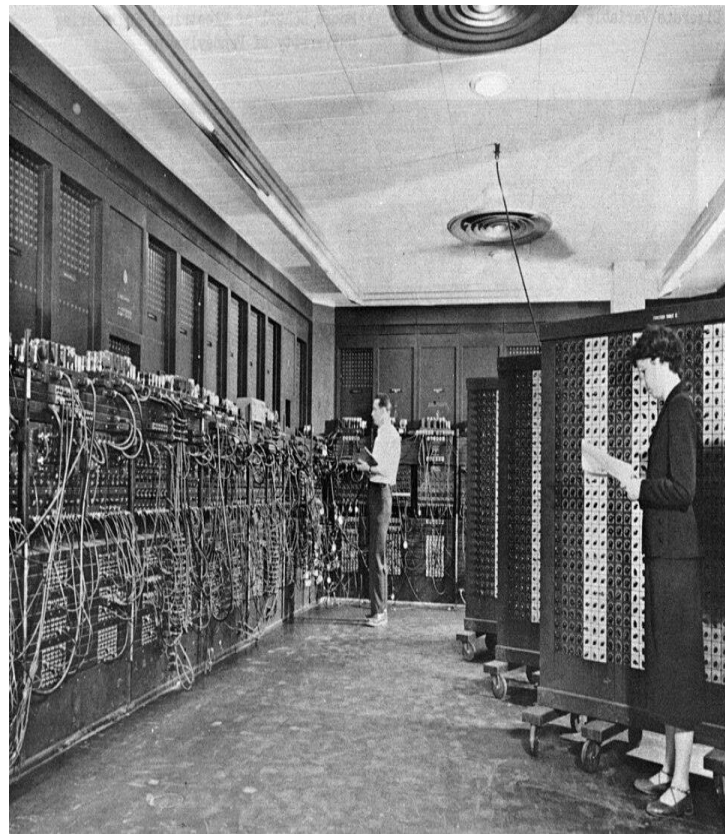
Range table for 3-inch field gun.

Range.	Angle of depart- ure.	Angle of depart- ure.	Angle of eleva- tion.	One minute, in yards of range.	One mil, in yards of range.	ΔX for $\pm$ 10 f. s. M. V.
<i>Yds.</i>	<i>° ' "</i>	<i>Mils.</i>	<i>° ' "</i>			<i>Yds.</i>
100	0 05.9	1.7	0 00.2	16.7	56	1.08
200	0 11.9	3.5	0 06.2	15.6	52	1.9
300	0 18.3	5.4	0 13.6	15.2	50	2.8
400	0 24.9	7.4	0 19.3	14.5	48	3.7
500	0 31.9	9.5	0 26.3	13.9	46	4.6



ENIAC and the Bomb (1945)

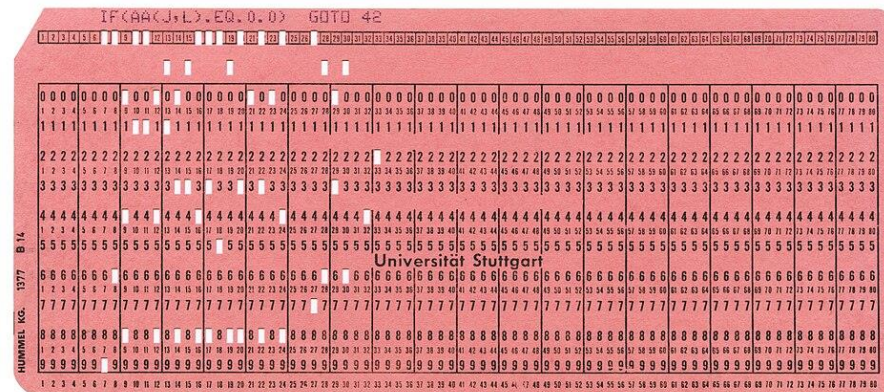
- The Army still needed trajectories faster than humans could compute.
New weapons, new theaters, hundreds of table entries per gun.
- Army Ordnance Corps funds ENIAC project
 - 50-foot room. 30 tons. 18,000 vacuum tubes. 150 kilowatts.
 - Switching it on dimmed the lights in West Philadelphia.
 - Programming meant rewiring - Six women from the human computing pool
 - 5,000 additions per second.
 - Same algorithm as the women with pencils. Same algorithm (Euler), just faster.
- The first real job isn't firing tables.
Von Neumann redirects it before it's finished:
 - Can a hydrogen bomb work? Six weeks of computation. One million punch cards.
 - The mathematical formulation: A system of coupled differential equations describing, at each point along the cylinder as a function of time:
 - Hydrodynamics — compression and shock wave propagation
 - Temperature evolution
 - Thermonuclear reaction rates
 - Radiation and energy transport





FORTRAN — Teaching Machines to Read Math (1957)

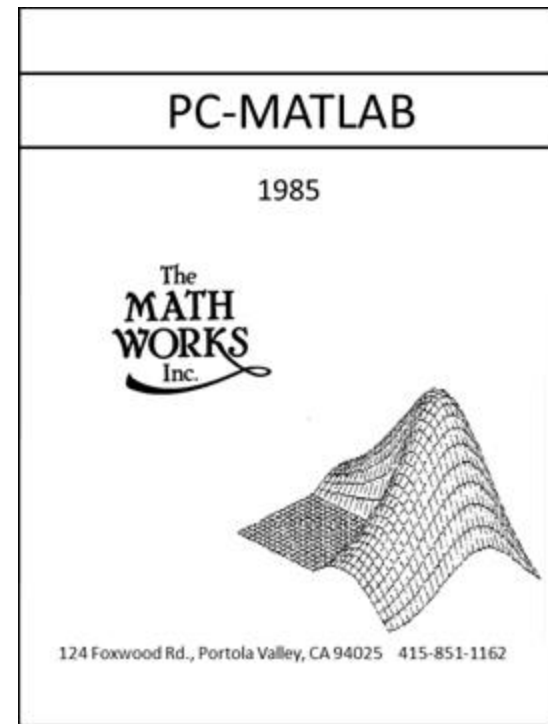
- The problem: the people who needed to solve differential equations weren't programmers. Programmers who understood the hardware couldn't formulate the physics.
 - Writing a single ballistic trajectory in machine code took weeks.
- John Backus, IBM, 1953:
 - *Let the machine read something close to mathematical notation.*
 - The received wisdom: no compiler could ever produce code as efficient as a skilled human programmer.
- MATLAB's core numerical routines are built on two legendary FORTRAN libraries:
 - LAPACK (Linear Algebra PACKage) — matrix operations, eigenvalues, linear solvers. MATLAB calling FORTRAN code written in the 1970s–90s.
 - BLAS (Basic Linear Algebra Subprograms) — the lower-level building blocks LAPACK itself calls. Matrix multiplication, dot products, vector scaling.





MATLAB (1984)

- The problem: FORTRAN libraries were powerful but inaccessible. Numerical analysis courses spent more time on software mechanics than mathematics.
- Cleve Moler, University of New Mexico, 1978:
 - Wants his students to use the best linear algebra libraries without writing FORTRAN.
 - Writes a thin interactive interface over a weekend. Matrices as first-class objects. Libraries one function call away.
 - Gives it away free on tape.
- 1984: Jack Little rewrites it in C, adds graphics, co-founds MathWorks with Moler.
 - Same idea as FORTRAN — eliminate the translation layer between the person with the problem and the machine that solves it.
 - FORTRAN let physicists stop writing machine code.
 - MATLAB let engineers stop writing FORTRAN.

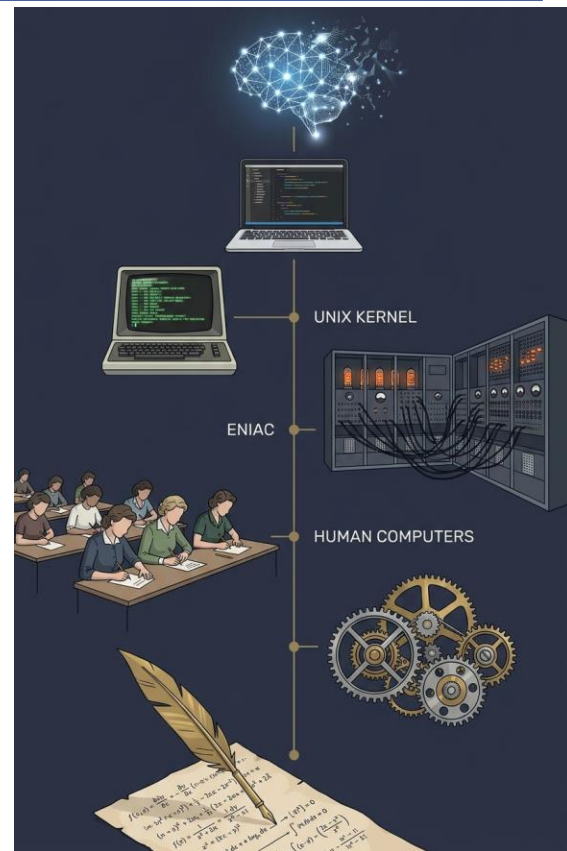


[MATLAB History, PC-MATLAB Version 1.0 — Cleve's Corner](#)



The Next Abstraction (2045)

- 300 years. One pattern.
 - Every generation inherited a translation problem — between the person with the physics and the machine that could compute it. Every generation solved it by building a higher layer.
 - Calculus abstracted geometry.
 - FORTRAN abstracted machine code.
 - MATLAB abstracted FORTRAN.
 - Machine learning abstracts the solver.
- 2031: The European Centre for Medium-Range Weather Forecasts retires its last hand-coded FORTRAN integrator. The replacement was trained on 80 years of atmospheric data. It was never told Newton's second law.
- 2038: A foundation model trained on declassified WWII firing tables — the ones computed by hand at Penn — proposes a correction to ballistic drag coefficients. The correction is measurable. The paper is accepted.
- 2045: A naval engineering student in 2045 describes a ship's dynamics in plain language and receives a validated simulation. They don't write a rate function. They don't choose a solver. They don't know what FORTRAN is.
 - Euler's algorithm is probably still running somewhere. It's just very far down.





Why Differential Equations Invented the Computer

- Every major leap in computing was either driven by the need to solve ODEs or immediately applied to solving them:
 - Babbage — navigational and astronomical tables
 - Penn — 100 women spent the war integrating trajectories by hand
 - ENIAC — built for firing tables; first real job was a thermonuclear ODE
 - FORTRAN (1957) — let scientists write math instead of assembly
 - MATLAB (1984) — let engineers skip FORTRAN
- CUDA (2007) — put thousands of parallel cores in the hands of scientists; the same ODE, solved simultaneously across a million grid points
 - Machine learning — the solver learns the physics; the translation layer disappears entirely (maybe?)
- The physical world is continuous. Computers are discrete.
 - Bridging that gap — turning a differential equation into a finite sequence of arithmetic steps — is the fundamental problem of scientific computing.